

P2809R0

Trivial infinite loops are not Undefined Behavior

Published Proposal, 2023-03-14

This version:

<http://wg21.link/P2809r0>

Author:

[JF Bastien](#) (Woven by Toyota)

Audience:

SG1, SG22, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P2809r0.bs

Table of Contents

- 1 **Introduction**
- 2 **Standards**
- 3 **History**
- 4 **Rationale**
- 5 **Wording**
- 6 **Alternatives**
 - 6.1 `yield`
 - 6.2 Early break
 - 6.3 Execution agent

References

Informative References

§ 1. Introduction

C and C++ diverge in their definition of forward progress guarantees, and have done so since C++11 and C11 added a memory model and acknowledged that threads exist. Both committees have been working together to reduce needless differences. [\[P2735r0\]](#) and [\[N2644\]](#) perform such harmonization.

C does not make iteration statements whose controlling expression is a constant expression Undefined Behavior, whereas C++ does. C++ implementations therefore assume that even "trivial" infinite loops must terminate.

This is unfortunate because using an infinite loop is a common idiom in low-level programming, particularly when a bare-metal system or kernel must halt progress. It would be easy for programmers to satisfy the C++ requirements on infinite loops, and it would be easy for compilers to diagnose "invalid" infinite loops. Neither programmers nor implementations have done so. Instead, programmers decry C++'s obtuseness, and implementations upset developers by optimizing away their infinite loops (marking them as unreachable, leading to surrounding code being optimized away).

For example, this code prints `Hello, World!` in recent versions of clang ([see example](#)):

```
#include <iostream>

int main() {
    while (true)
        ;
}

void unreachable() {
    std::cout << "Hello world!" << std::endl;
}
```

It is easy for the C++ Standards Committee to instead align with C, and allow "trivial" infinite loops as C does. However, care must be taken when doing so. C++ has good reasons to specify that infinite loops are undefined behavior: it is desirable to express forward progress guarantees in a programming language because it unlocks a design space in concurrency and parallelism software and hardware design. An 80 minute deep-dive in this topic is available in [\[ForwardProgress\]](#). A definition of "trivial" which continues supporting this design space is important to reach.

Other programming languages, compilers, and hardware explicitly or de-facto rely on the C++ specification of its abstract machine, including the behavior infinite loops. Some by borrowing from C++'s specification, others by simply using compilers that implement C++ as their optimizer (see [\[LLVMProgressDiscussion\]](#), and [a Rust bug of this shape](#)), and yet others by being designed around C++ programs. It is therefore likely that a broader specification for an abstract machine, outside of C++, is needed to truly resolve problems as those tackled in this paper.

§ 2. Standards

Since C++11, the Standard's forward progress guarantees have been defined in **[intro.progress]** as follows:

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- perform an access through a `volatile` glvalue, or
- perform a synchronization operation or an atomic operation.

[*Note*: This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. —*end note*]

This forward progress guarantee applies to the following iteration statements:

```
while ( condition ) statement
do statement while ( expression ) ;
for ( init-statement conditionopt ; expressionopt ) statement
for ( init-statementopt for-range-declaration : for-range-initializer ) statement
```

Since C11, the Standard's forward progress guarantees have been defined in **Iteration statements** (section 6.8.5 of C17) as follows:

An iteration statement may be assumed by the implementation to terminate if its controlling expression is not a constant expression[†], and none of the following operations are performed in its body, controlling expression or (in the case of a `for` statement) its *expression-3*[‡]:

- input/output operations
- accessing a `volatile` object
- synchronization or atomic operations.

[†] An omitted controlling expression is replaced by a nonzero constant, which is a constant expression.

[‡] This is intended to allow compiler transformations such as removal of empty loops even when termination cannot be proven.

This forward progress guarantee applies to the follow iteration statements:

```
while ( expression ) statement
do statement while ( expression ) ;
for ( expressionopt ; expressionopt ; expressionopt ) statement
for ( declaration expressionopt ; expressionopt ) statement
```

By omission, iteration statements are undefined if the controlling expression does not meet the criteria above.

§ 3. History

C99 6.8.5p4 states:

An iteration statement causes a statement called the loop body to be executed repeatedly until the controlling expression compares equal to 0.

C99 therefore disallows compilers from assuming that loops will eventually terminate.

C++11 added a full memory model to C++, allowing C++ to speak of threads and inter-thread communications in a useful manner. This forward progress guarantee does away with the C99 guarantee above. A key to concurrency and parallelism is understanding when forward progress is guaranteed, because without forward progress some concurrent and parallel algorithms do not work.

For example, a pair of producer / consumer threads will never consume data if the producer does not have forward progress.

This memory model work was imported into the C11 standard through [\[N1349\]](#). There were discussions on the topic of forward progress:

- [\[N1509\]](#) proposed eliminating the paragraph on forward progress from C1X.
- [\[N1528\]](#) responded to N1509, explaining the C1X wording.

These two papers were discussed, and WG14 took the following decision:

ACTION Douglas to work with Larry to come up with the proposed words are:

Change 6.8.5p6 as follows:

An iteration statement whose controlling expression is not a constant expression (or omitted), that performs no input/output operations, does not access `volatile` objects, and performs no synchronization or atomic operations in its body, controlling expression, or (in the case of a `for` statement) its *expressionX3*, may be assumed by the implementation to terminate.

Decision: Adopt N1509, as modified above, to the C1X WP. result: 14 favor, 0 neutral, 1 against

§ 4. Rationale

The forward progress guarantees were added to C++11 and C11 along with a memory model. This addition of forward progress guarantees reflected existing practice in compilers, and did not simply serve concurrency and parallelism goals. Indeed, compilers had been assuming that loops can terminate to enable optimizations that transform loops and reorder store instructions and other side-effects. Proving that loops terminate is generally equivalent to solving the halting problem. The compilers were therefore disregarding the C99 wording which said that loops terminate when their controlling expression becomes `0`, and were therefore non-conformant.

The standards added the forward progress guarantees to change an optimization problem from "solve the halting problem" to "there will be observable side effects in the forms of termination, I/O, `volatile`, and/or atomic synchronization, any other operation can be reordered". The former is generally impossible to solve, whereas the latter is eminently tractable.

Forward progress guarantees therefore helped program concurrency and parallelism, and standardized existing practice which unlocked performance in compiler optimizations.

However, developers do sometimes rely on infinite loops, for valid reasons. When they do so, the loops have controlling expressions which are constant expressions. Such controlling expressions are known at compile-time by definition, and are therefore easy for a compiler to detect, and mark a loop as not open to optimizations which assume termination.

This change does not affect:

- Loops whose controlling expressions are not constant expressions, which are the loops that benefit from optimizations related to termination guarantees;
- Other forms of backwards control flow such as `goto`, `setjmp/longjmp`, infinite tail recursion;
- Forward progress guarantees for concurrent and/or parallel programs (unless the execution agents have a trivial infinite loop).

§ 5. Wording

Modify **[intro.progress]** as follows:

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- perform an access through a `volatile` glvalue, ~~or~~
- perform a synchronization operation or an atomic operation ~~:~~, or
- execute an iteration statement (*[stmt.iter]*) whose controlling expression is a converted constant expression (*[expr.const]*) that converts to `true`, where the controlling expression is the condition of a `while` or `for` loop, or the expression of a `do/while` loop.

[Note: This is intended to allow compiler transformations such as removal, merging, and reordering of non-trivial empty loops, even when termination cannot be proven. —end note]

§ 6. Alternatives

An advantage of the above proposal is that it matches the C semantics. Any alternative proposal would require changes to C, and might require changes to other languages, compilers, or hardware which rely on the behavior being changed. It should be pointed out that C technically does not have forward progress guarantees, but hints at maybe having some guarantees. C also doesn't reach the same conclusion about infinite loops using `goto` or `setjmp/longjmp`.

§ 6.1. `yield`

A problem with the above resolution is that some GPU hardware would halt forward progress on *all* warps in a thread block if one of the warps were to have such an infinite loop. This behavior is acceptable while infinite loops are Undefined Behavior, An alternative would be to specify that such "trivial" infinite loops are equivalent to calling `std::this_thread::yield()` on the backedge. Care should then be taken to specify when an implicit `yield` is added depending on the loop body.

The above approach with `yield` also resolves problems pertaining to code motion which the above resolution does not: is a compiler allowed to hoist code from below a "trivial" infinite loop to above it? The clear answer with the `yield` approach is "no, code cannot be hoisted".

§ 6.2. Early break

An issue with the proposed resolution (and the C status quo) is that this loop:

```
while (cond) {  
    // ...  
}
```

is not equivalent to:

```
while (true) {  
    if (!cond)  
        break;  
}
```

The same applies for any loop which has a constant expression controlling expression that converts to `true`, and has a `return`, `longjmp`, or (potentially nested) `throw`.

§ 6.3. Execution agent

Another alternative would be to tie semantics of "trivial" infinite loops to the progress guarantees of the execution agent (i.e. for concurrent progress only).

§ References

§ Informative References

[ForwardProgress]

Olivier Giroux. Forward Progress in C++. 2022-09-12. URL: <https://www.youtube.com/watch?v=CuWM-OrPitw>

[LLVMProgressDiscussion]

Nikita Popov. [LangRef] Document forward-progress requirement (Abandoned). 2019-08-04. URL: <https://reviews.llvm.org/D65718>

[N1349]

Clark Nelson; Hans-J. Boehm; Lawrence Crowl. Parallel memory sequencing model proposal. 2009-02-23. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1349.htm>

[N1509]

Douglas Walls. Optimizing away infinite loops. 2010-09-02. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1509.pdf>

[N1528]

Hans Boehm. Why undefined behavior for infinite loops?. 2010-10-27. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1528.htm>

[N2644]

Jens Gustedt. Programming languages — a common C/C++ core specification. 2021-01-20. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2644.pdf>

[P2735r0]

Aaron Ballman. C xor C++ Programming. 5 December 2022. URL: <https://wg21.link/p2735r0>